

A PRAGMATIC INTRODUCTION TO CATEGORY THEORY

@DANIELASFREGOLA

CODE MESH 2017

github.com/DanielaSfregola/tutorial-cat

HELLO00000

- > EX JAVA DEVELOPER
- > OOP BACKGROUND
- > I AM NOT A MATHEMATICIAN 🤔



I AM NOT A MATHEMATICIAN



**YOU DO NOT NEED TO KNOW
CATEGORY THEORY
TO WRITE
GOOD CODE**

**YOU DO NOT NEED TO KNOW
CATEGORY THEORY
TO WRITE
FUNCTIONAL CODE**

CATEGORY THEORY

DEEPER UNDERSTANDING ON OUR CODE

HOW DO WE
REASON ?

COMPOSITION
ABSTRACTION

CATEGORY THEORY

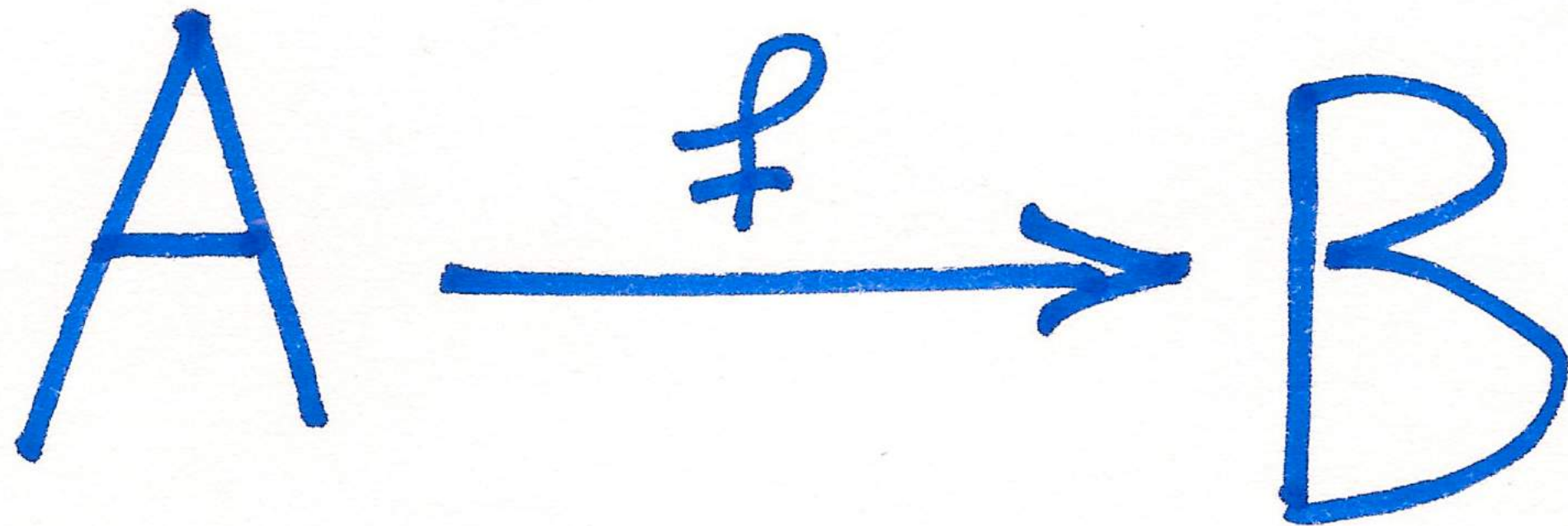
HOW THINGS COMPOSE

ARROW THEORY

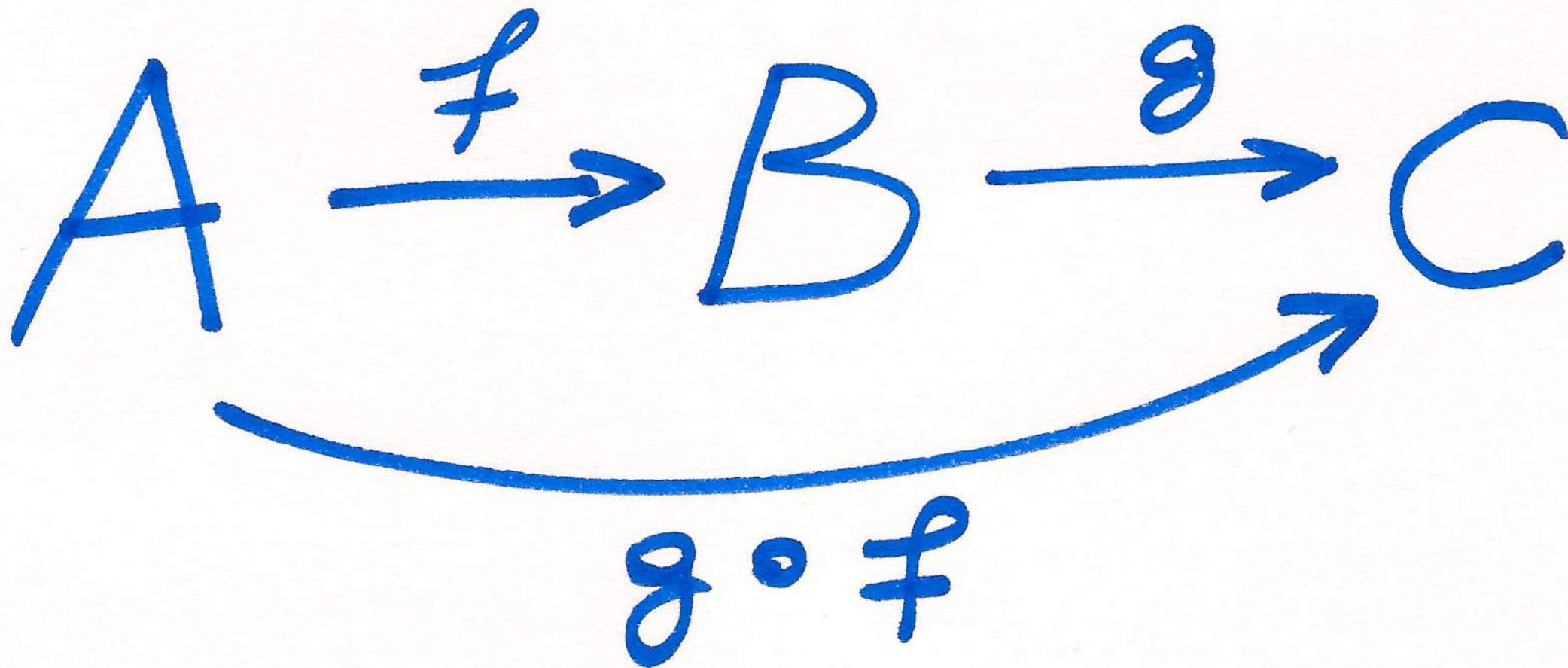
~~CATEGORY THEORY~~

HOW THINGS COMPOSE

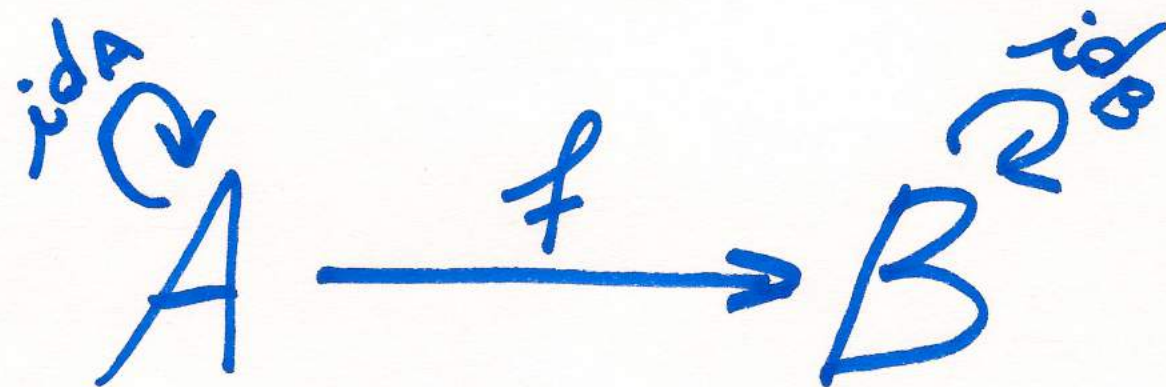
WHAT IS A CATEGORY?



COMPOSITION LAW

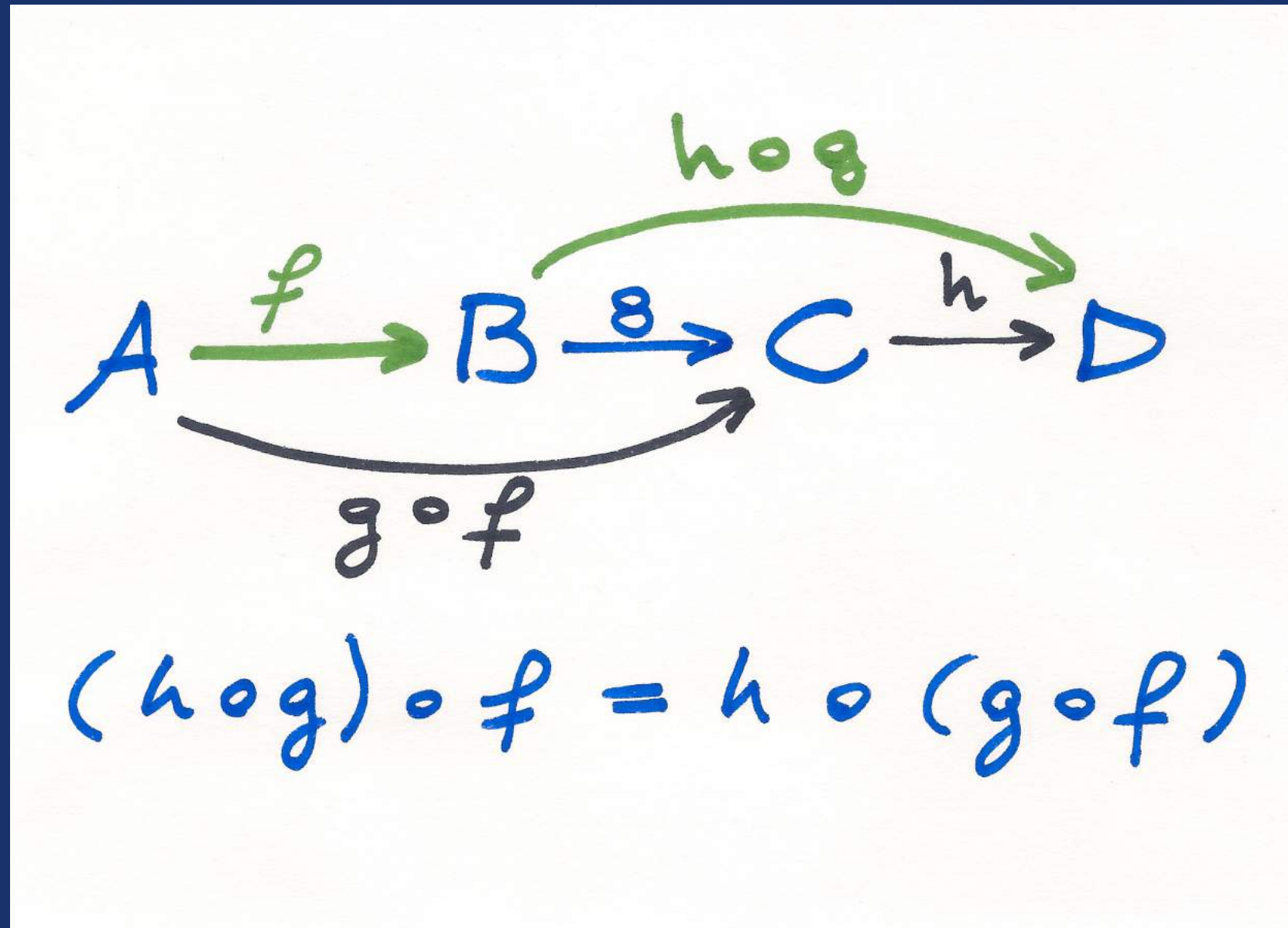


IDENTITY LAW



$$f \circ id_A = f = id_B \circ f$$

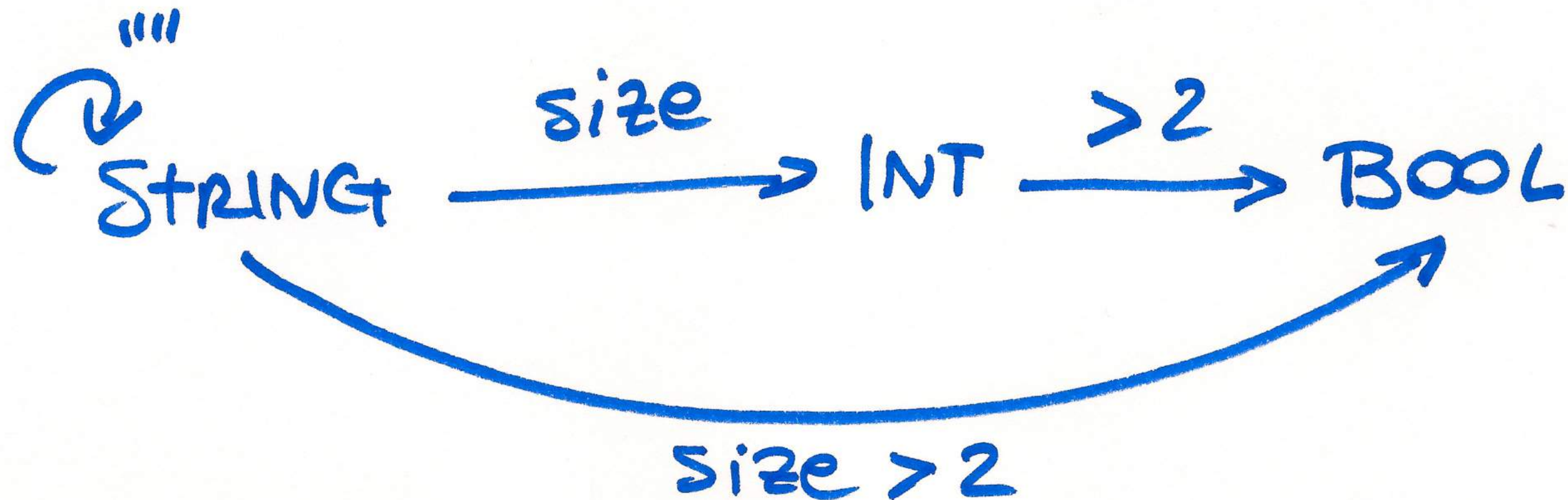
COMPOSITION + ASSOCIATIVITY



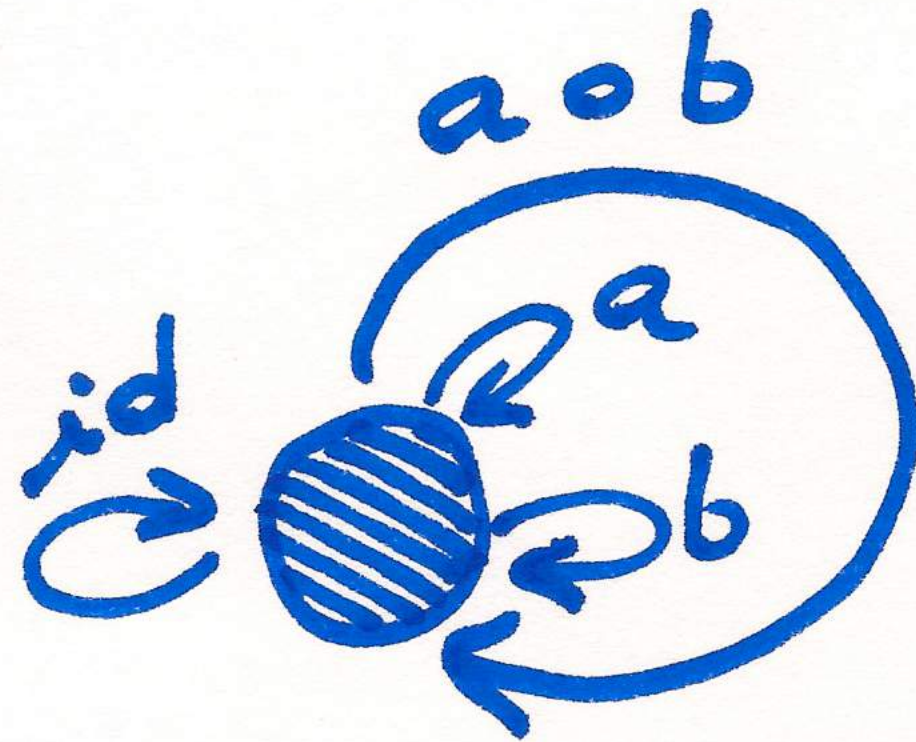
CATEGORY'S RULES

- > IDENTITY
- > COMPOSITION
- > ASSOCIATIVITY

A PRACTICAL EXAMPLE



CATEGORY WITH 1 OBJECT



$$id \circ a = a = a \circ id$$

CATEGORY WITH 1 OBJECT

=

MONOID

MONOID'S RULES

IDENTITY

$$n \circ id == id \circ n == n$$

COMPOSITION

$$\text{forall } x, y \Rightarrow x \circ y$$

ASSOCIATIVITY

$$x \circ (y \circ z) == (x \circ y) \circ z$$

SCALACHECK FTW!

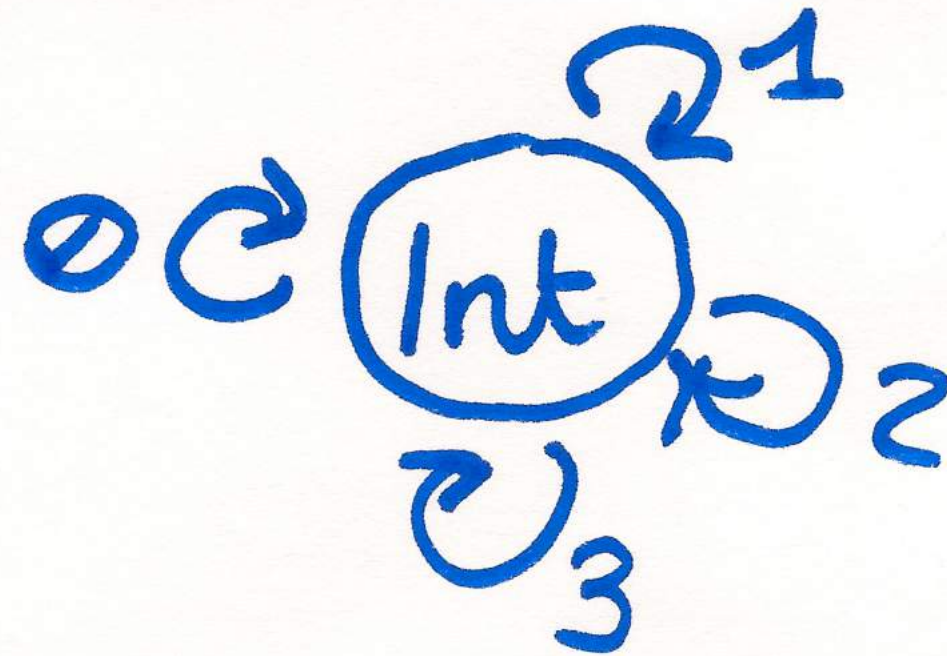
MonoidSpec

```
// n o id == id o n == n
property("identity") = forAll { n: A =>
  monoid.compose(n, id) == n &&
  monoid.compose(id, n) == n
}

// forall x, y => x o y
property("composition") = forAll { (x: A, y: A) =>
  monoid.compose(x, y).isInstanceOf[A]
}

// x o (y o z) == (x o y) o z
property("associativity") = forAll { (x: A, y: A, z: A) =>
  val xY = monoid.compose(x, y)
  val yZ = monoid.compose(y, z)
  monoid.compose(xY, z) == monoid.compose(x, yZ)
}
```

A PRACTICAL EXAMPLE



$op = +$
 $id = 0$

IDENTITY

$$1 + 0 = 1 = 0 + 1$$

$$2 + 0 = 2 = 0 + 2$$

COMPOSITION

$$1 + 1 = 2$$

$$1 + 2 = 3$$

MONOID

```
trait Monoid[A] {  
  def identity: A  
  def compose(x: A, y: A): A  
}
```

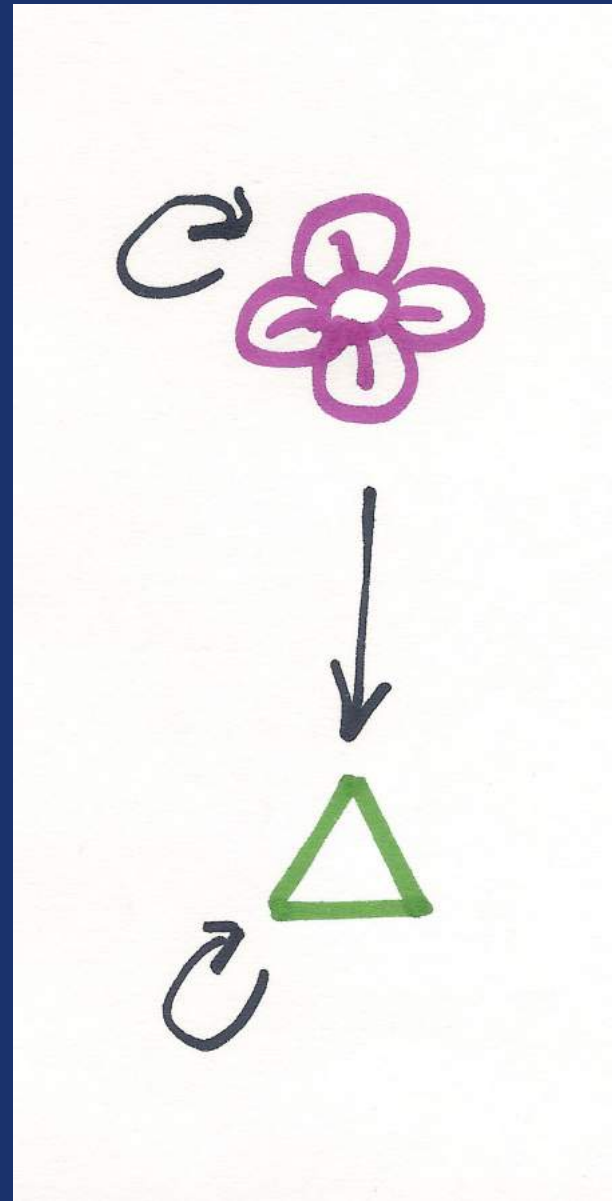

MONOID INSTANCES (1)

```
implicit val intMonoid: Monoid[Int] =  
  new Monoid[Int] {  
    def compose(x: Int, y: Int): Int = x + y  
    def identity: Int = 0  
  }
```

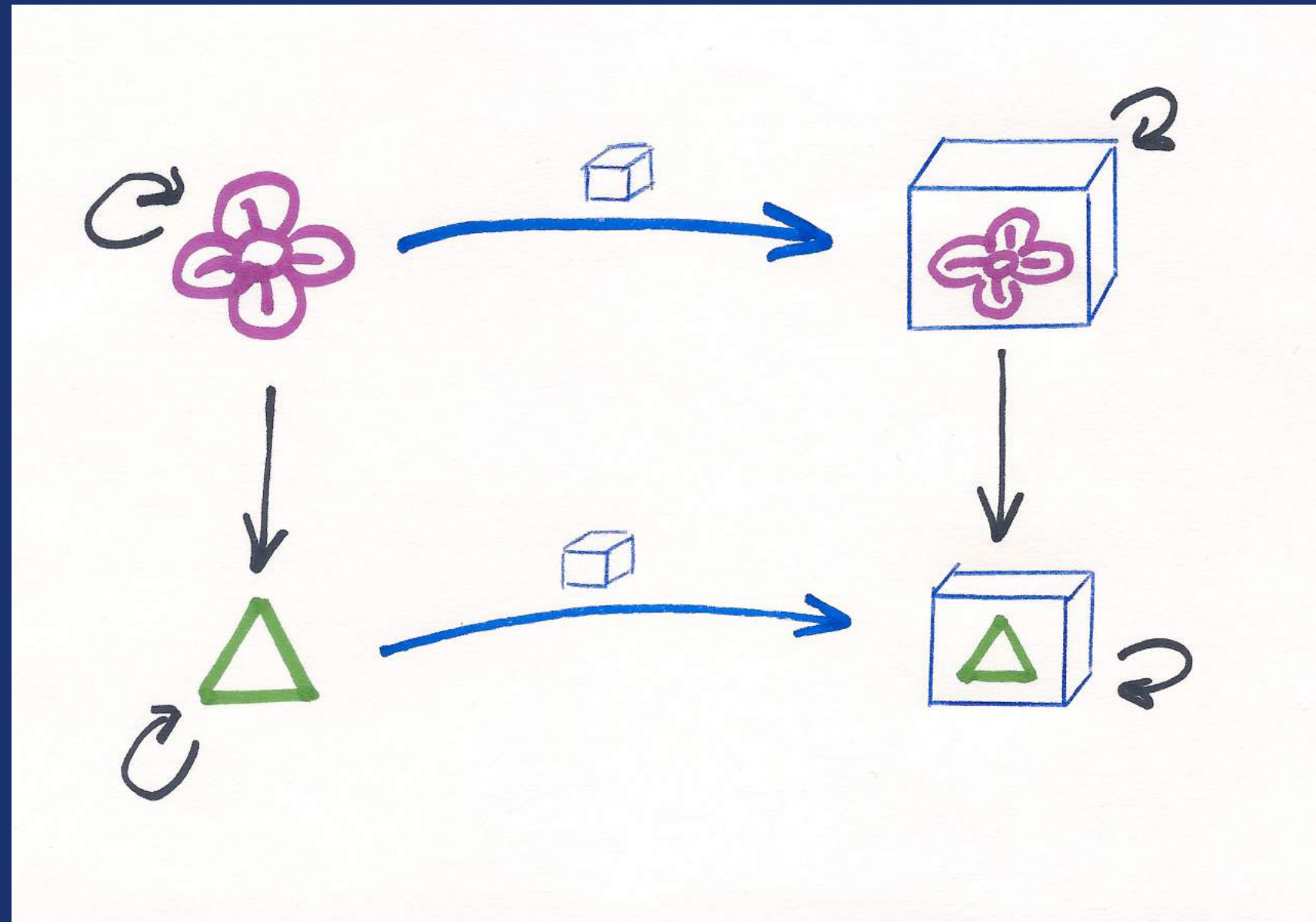
MONOID INSTANCES (2)

```
implicit val stringMonoid: Monoid[String] =  
  new Monoid[String] {  
  
    def compose(x: String, y: String): String = x + y  
  
    def identity: String = ""  
  }
```

CATEGORY WITH 1+ OBJECT



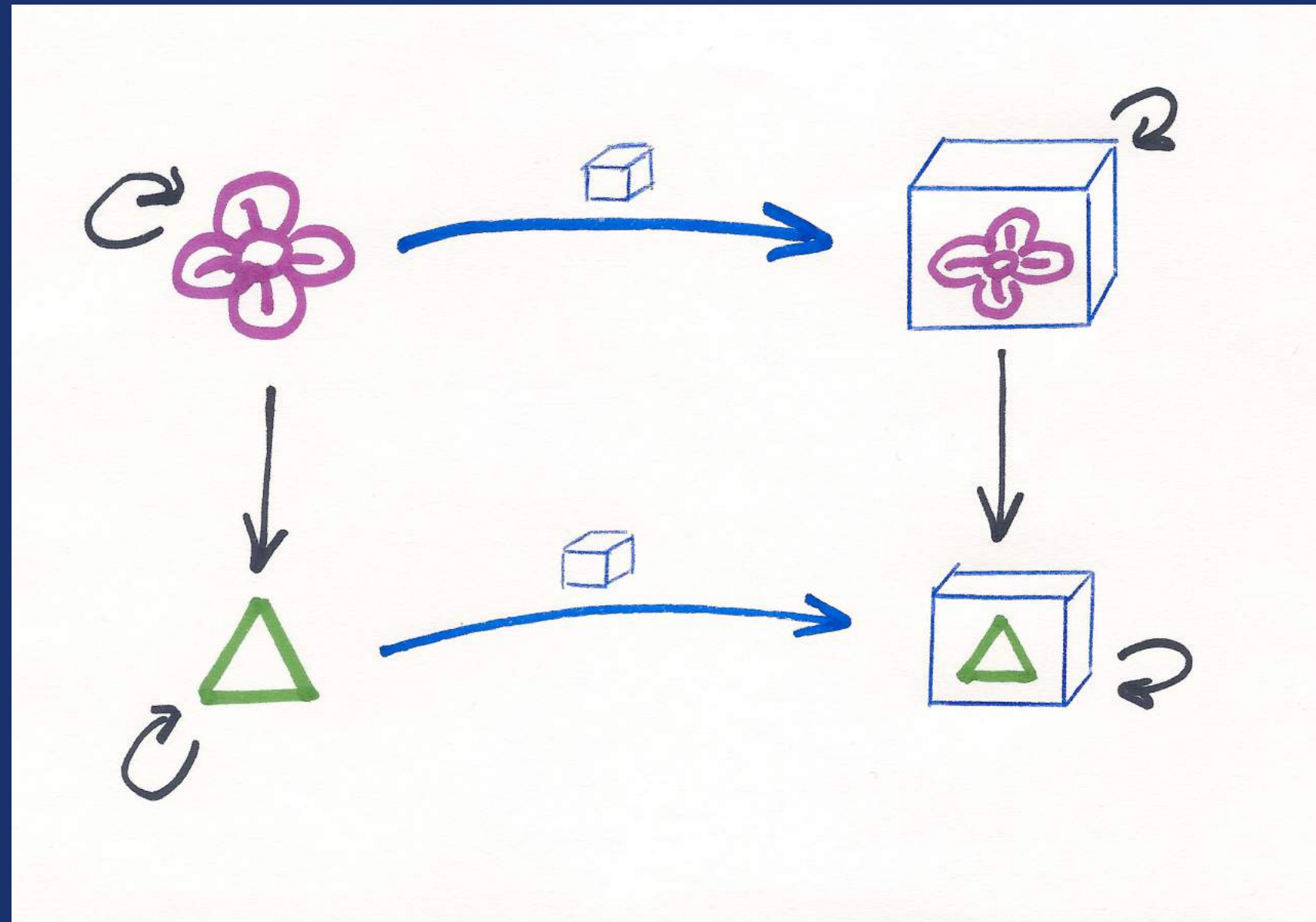
CATEGORY IN A BOX



CATEGORY IN A BOX

- > OBJECTS ARE IN A BOX
- > ALL THE ARROWS ARE MAPPED

LIFTING: CONTEXT VS CONTENT



EXAMPLE OF BOXES

- > Option
- > Future
- > Try
- > List
- > Either

CATEGORY IN A BOX

=

FUNCTOR

FUNCTOR'S RULES

IDENTITY

$$\text{map}(\text{id}) == \text{id}$$

COMPOSITION

$$\text{map}(g \circ f) == \text{map}(g) \circ \text{map}(f)$$

ASSOCIATIVITY

$$\text{map}(h \circ g) \circ \text{map}(f) == \text{map}(h) \circ \text{map}(g \circ f)$$

SCALACHECK FTW!

FunctorSpec

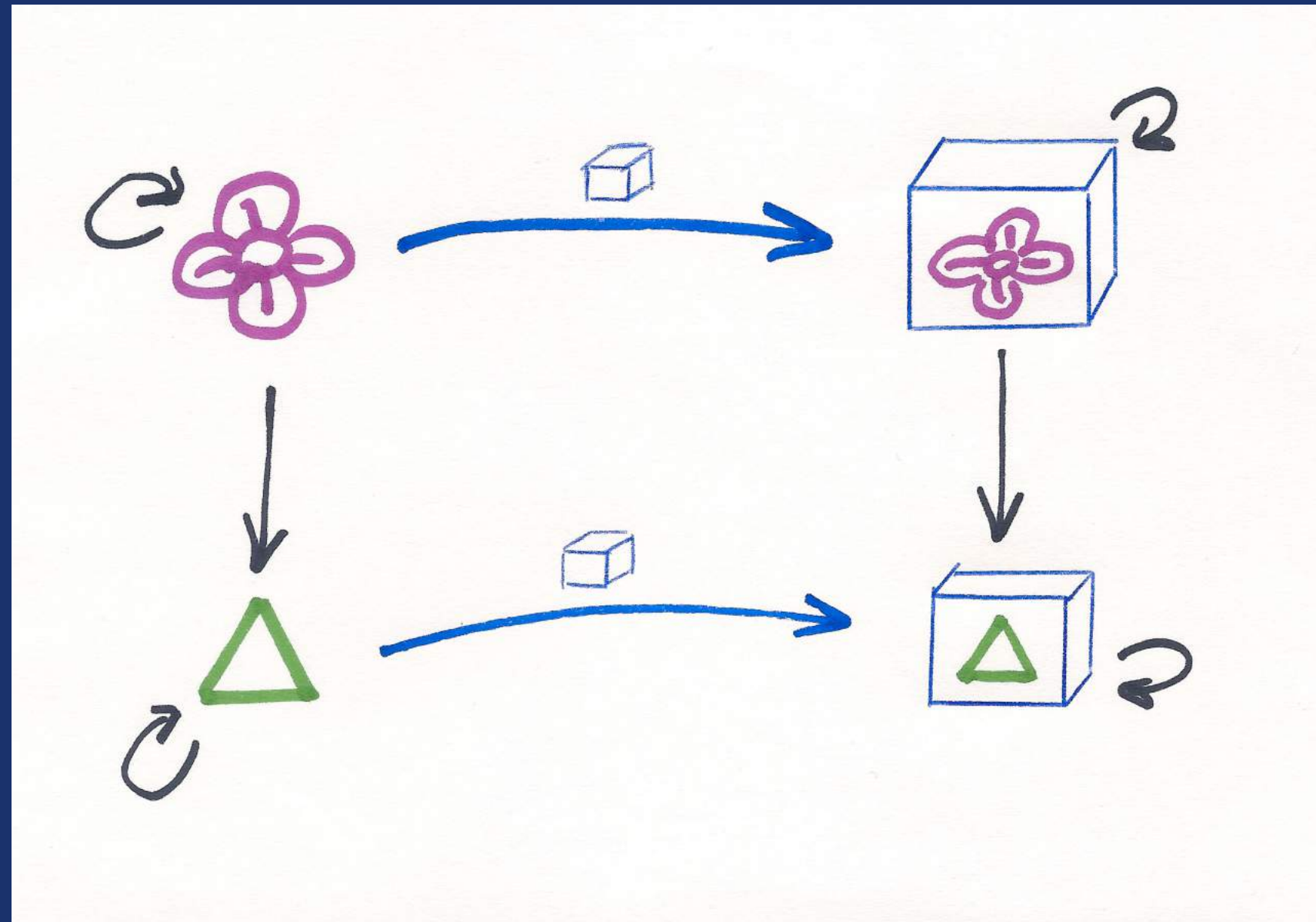
```
// map_id == id
property("identity") = forAll { box: Box[A] =>
  map(box)(identity) == box
}

// map_(g o f) == (map_g) o (map_f)
property("composition") = forAll { boxA: Box[A] =>
  val fG = f andThen g
  val mapFG: Box[A] => Box[C] = map(_)(fG)
  mapFG(boxA) == (mapF andThen mapG)(boxA)
}

// map_(h o g) o map_f == map_h o map_(g o f)
property("associativity") = forAll { boxA: Box[A] =>
  val fG = f andThen g
  val mapFG: Box[A] => Box[C] = map(_)(fG)
  val gH = g andThen h
  val mapGH: Box[B] => Box[D] = map(_)(gH)

  (mapF andThen mapGH)(boxA) == (mapFG andThen mapH)(boxA)
}
```

LIFTING: CONTEXT VS CONTENT



FUNCTOR

```
class Functor[Box[_]] {  
  
  def map[A, B](boxA: Box[A])  
    (f: A => B): Box[B]  
  
}
```


MAYBE

```
sealed abstract class Maybe[+A]
```

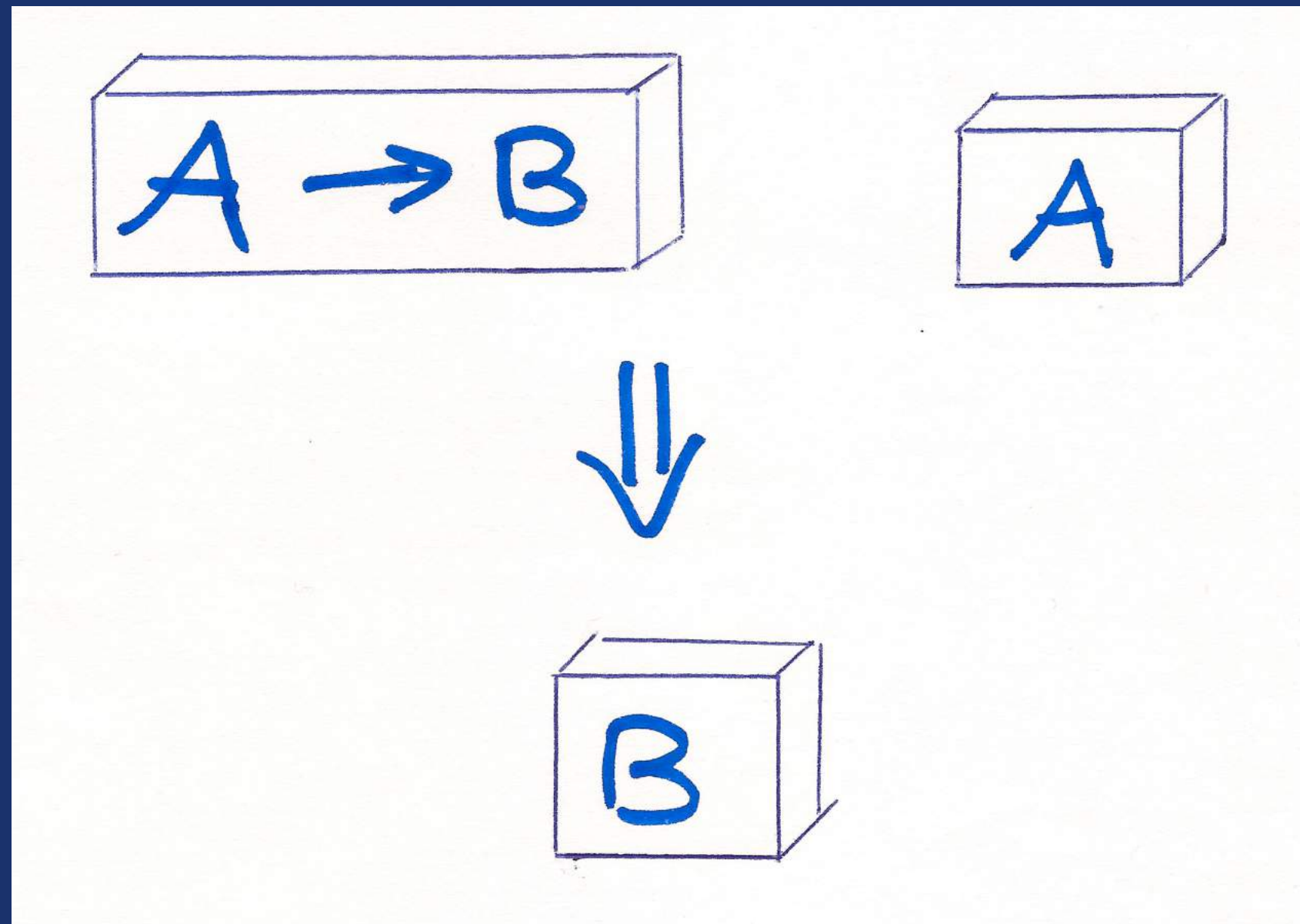
```
final case class Just[A](a: A) extends Maybe[A]
```

```
case object Empty extends Maybe[Nothing]
```


FUNCTOR FOR MAYBE

```
implicit val maybeFunctor: Functor[Maybe] =  
new Functor[Maybe] {  
  
  override def map[A, B](boxA: Maybe[A])  
                        (f: A => B): Maybe[B] =  
    boxA match {  
      case Just(a) => Just(f(a))  
      case Empty   => Empty  
    }  
}
```

BOX FUNCTION + BOX VALUES



**COMBINE MORE BOXES
INTO ONE**

=

APPLICATIVE

APPLICATIVE'S RULES

- > IDENTITY
- > COMPOSITION
- > ASSOCIATIVITY
- > HOMOMORPHISM
- > INTERCHANGE

...AND MORE!

SCALACHECK FTW!

ApplicativeSpec extends FunctorSpec

```
// ap(id)(a) == a
property("identity") = forAll { box: Box[A] =>
  ap(pureIdentity)(box) == box
}

// ap(pure(f))(pure(a)) == pure(f(a))
property("homomorphism") = forAll { a: A =>
  ap(pureF)(pure(a)) == pure(f(a))
}

// {x => pure(x)}(a) == pure(a)
property("interchange") = forAll { a: A =>
  toPureA(a) == pure(a)
}

// pure(h o g o f) == ap(pure(h o g))(pure(f(a)))
property("composition") = forAll { a: A =>
  val gH = g andThen h
  val fGH = f andThen gH
  val pureGH = pure(gH)
  val pureFA = pure(f(a))
  pure(fGH(a)) == ap(pureGH)(pureFA)
}
```

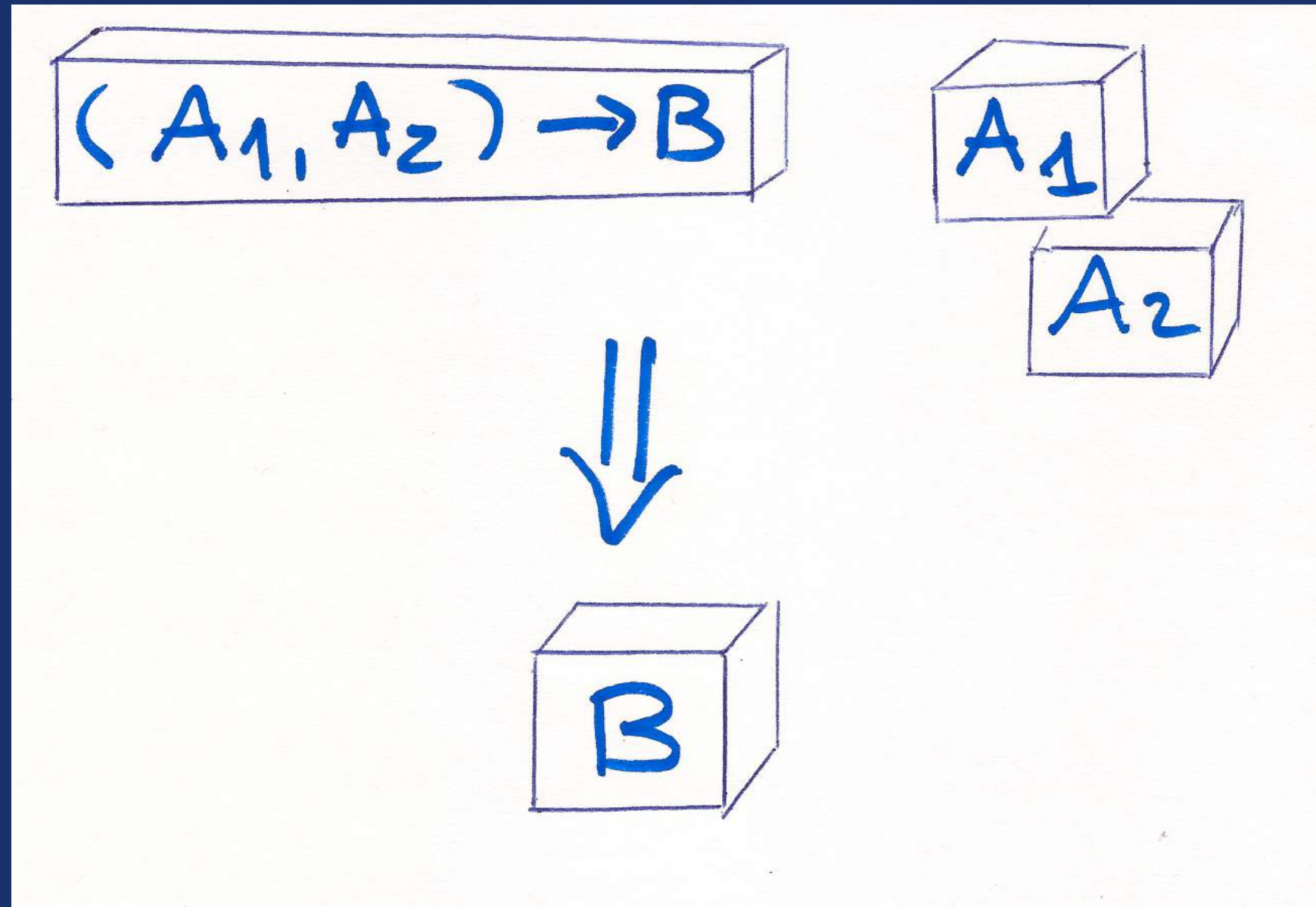
COMBINE BOXES TOGETHER

- HOW TO CREATE A NEW BOX
- HOW TO COMBINE THEIR VALUES TOGETHER

APPLICATIVE

```
class Applicative[Box[_]] extends Functor[Box] {  
  def pure[A](a: A): Box[A]  
  
  def ap[A, B](boxF: Box[A => B])(boxA: Box[A]): Box[B]  
  
  /**  
  
  def map[A, B](boxA: Box[A])(f: A => B): Box[B] =  
    ap[A, B](pure(f))(boxA)  
  
}
```


BOX FUNCTION + BOX VALUES



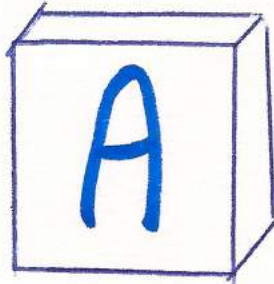
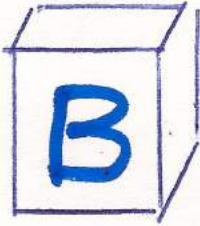
APPLICATIVE

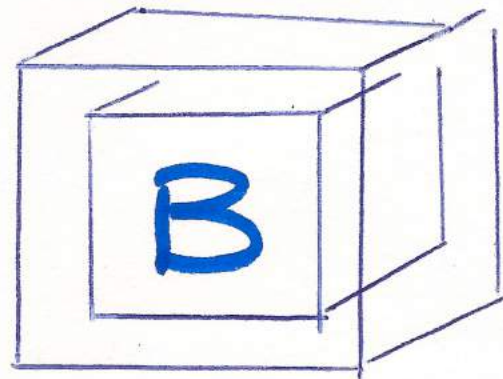
```
class Applicative[Box[_]] extends Functor[Box] {  
  def pure[A](a: A): Box[A]  
  
  def ap[A, B](boxF: Box[A => B])(value: Box[A]): Box[B]  
  
  def ap2[A1, A2, B](boxF: Box[(A1, A2) => B])  
    (value1: Box[A1], value2: Box[A2]): Box[B]  
  
  // up to 22 values!  
  // same for map  
}
```

APPLICATIVE FOR MAYBE

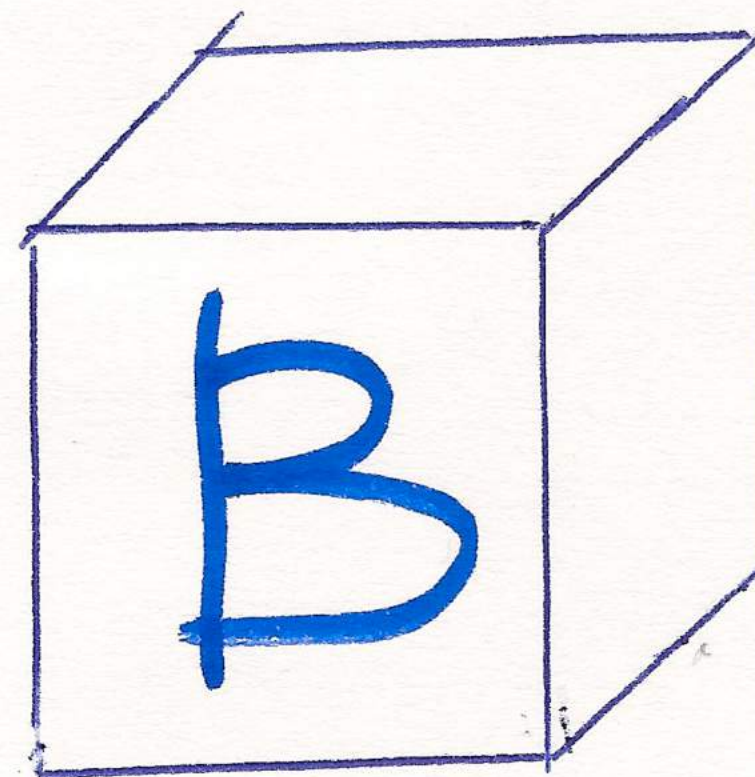
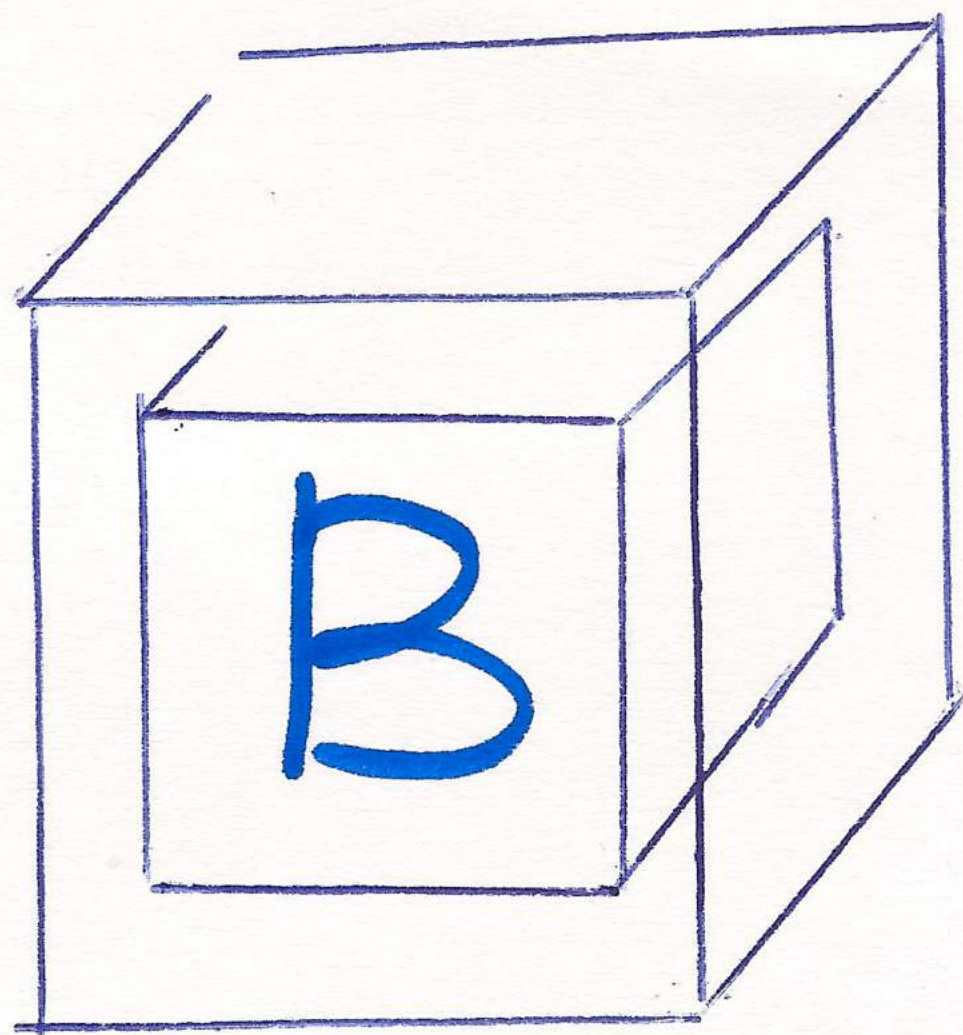
```
implicit val maybeApplicative: Applicative[Maybe] =  
new Applicative[Maybe] {  
  def pure[A](a: A): Maybe[A] = Just(a)  
  
  def ap[A, B](boxF: Maybe[A => B])(boxA: Maybe[A]): Maybe[B] =  
    (boxF, boxA) match {  
      case (Just(f), Just(a)) => pure(f(a))  
      case _ => Empty  
    }  
}
```

BOX IN A BOX

 . map (A → )



BOX IN A BOX



FUSE TWO BOXES
TOGETHER

=

MONAD

MONAD'S RULES

- > IDENTITY
- > COMPOSITION
- > ASSOCIATIVITY

SCALACHECK FTW!

MonadSpec extends ApplicativeSpec

```
// flatMap(pure(a))(f(a)) == f(a)
property("left identity") = forAll { a: A =>
  flatMap(pure(a))(toPureFa) == toPureFa(a)
}

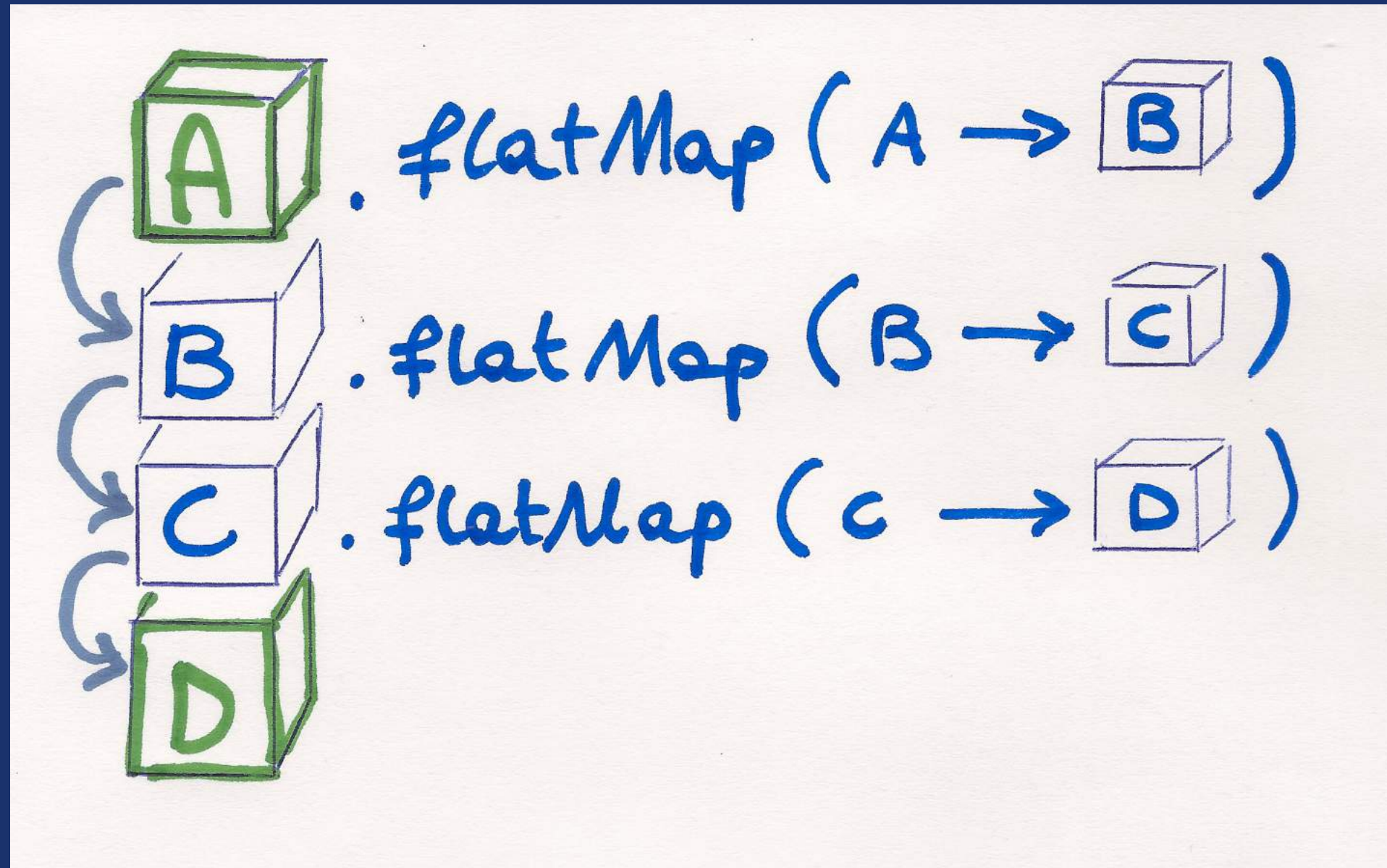
// flatMap(pure(a))(f(a)) == f(a)
property("right identity") = forAll { a: A =>
  flatMap(toPureFa(a))(pure) == toPureFa(a)
}

// pure(h o g o f) == ap(pure(h o g))(pure(f(a)))
property("associativity") = forAll { boxA: Box[A] =>
  val left: Box[C] = flatMap(flatMap(boxA)(toPureFa))(toPureGb)
  val right: Box[C] = flatMap(boxA)(a => flatMap(toPureFa(a))(toPureGb))
  left == right
}
```


MONAD (AS FUNCTOR)

```
class Monad[Box[_]] extends Functor[Box] {  
  
  // map from Functor  
  
  def flatten[A](bb: Box[Box[A]]): Box[A]  
  
  /** */  
  
  def flatMap[A, B](valueA: Box[A])  
    (f: A => Box[B]): Box[B] = {  
    val bb: Box[Box[B]] = map(valueA)(f)  
    bb.flatten  
  }  
}
```

BOXES IN A SEQUENCE



FOR-COMPREHENSION

```
val boxA: Box[A]  
def toBoxB: A => Box[B]  
def toBoxC: B => Box[C]  
def toBoxD: C => Box[D]
```

```
for {  
  a <- boxA  
  b <- toBoxB(a)  
  c <- toBoxC(b)  
  d <- toBoxD(c)  
} yield d
```

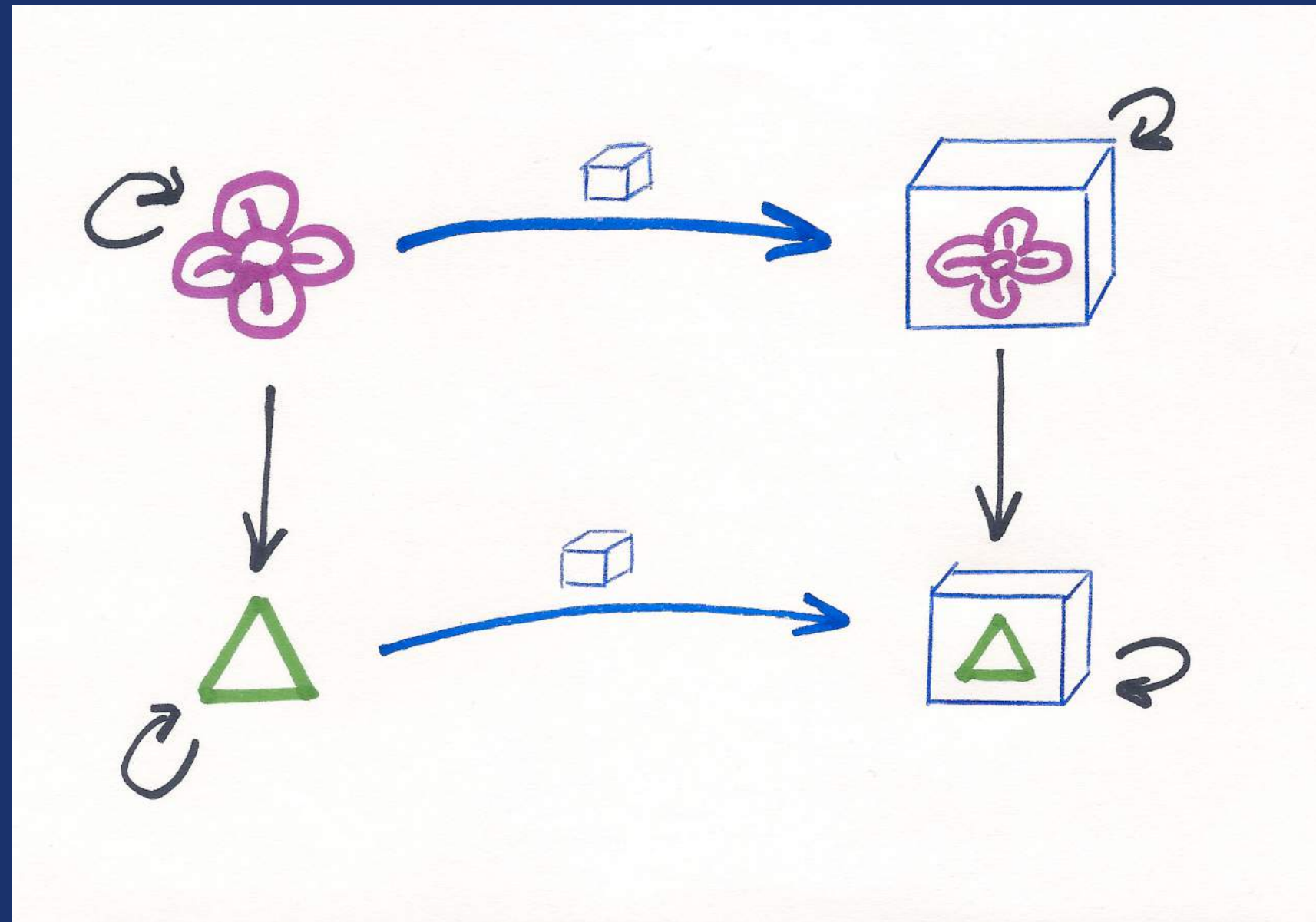
MONAD (AS APPLICATIVE)

```
trait Monad[Box[_]] extends Applicative[Box] {  
  
  // pure from Applicative  
  
  def flatMap[A, B](boxA: Box[A])(f: A => Box[B]): Box[B]  
  
  /** */  
  
  def flatten[A](boxBoxA: Box[Box[A]]): Box[A] =  
    flatMap(boxBoxA)(identity)  
  
  def ap[A, B](boxF: Box[A => B])(boxA: Box[A]): Box[B] =  
    flatMap(boxF)(f => map(boxA)(f))  
  
  def map[A, B](boxA: Box[A])(f: A => B): Box[B] =  
    flatMap(boxA)(a => pure(f(a)))  
  
}
```

MONAD FOR MAYBE

```
implicit val maybeMonad: Monad[Maybe] = new Monad[Maybe] {  
  
  def flatMap[A, B](boxA: Maybe[A])  
    (f: (A) => Maybe[B]): Maybe[B] =  
    boxA match {  
      case Just(a) => f(a)  
      case Empty => Empty  
    }  
  
  def pure[A](a: A): Maybe[A] =  
    maybeApplicative.pure(a)  
}
```

FUNCTOR VS ENDOFUNCTOR



MONAD IS
A MONOID
IN THE CATEGORY OF
ENDOFUNCTORS

MONAD

MONOID $\Rightarrow$ pure + flatten

ENDOFUNCTORS $\Rightarrow$ map

SUMMARY

CATEGORY THEORY >> how things compose

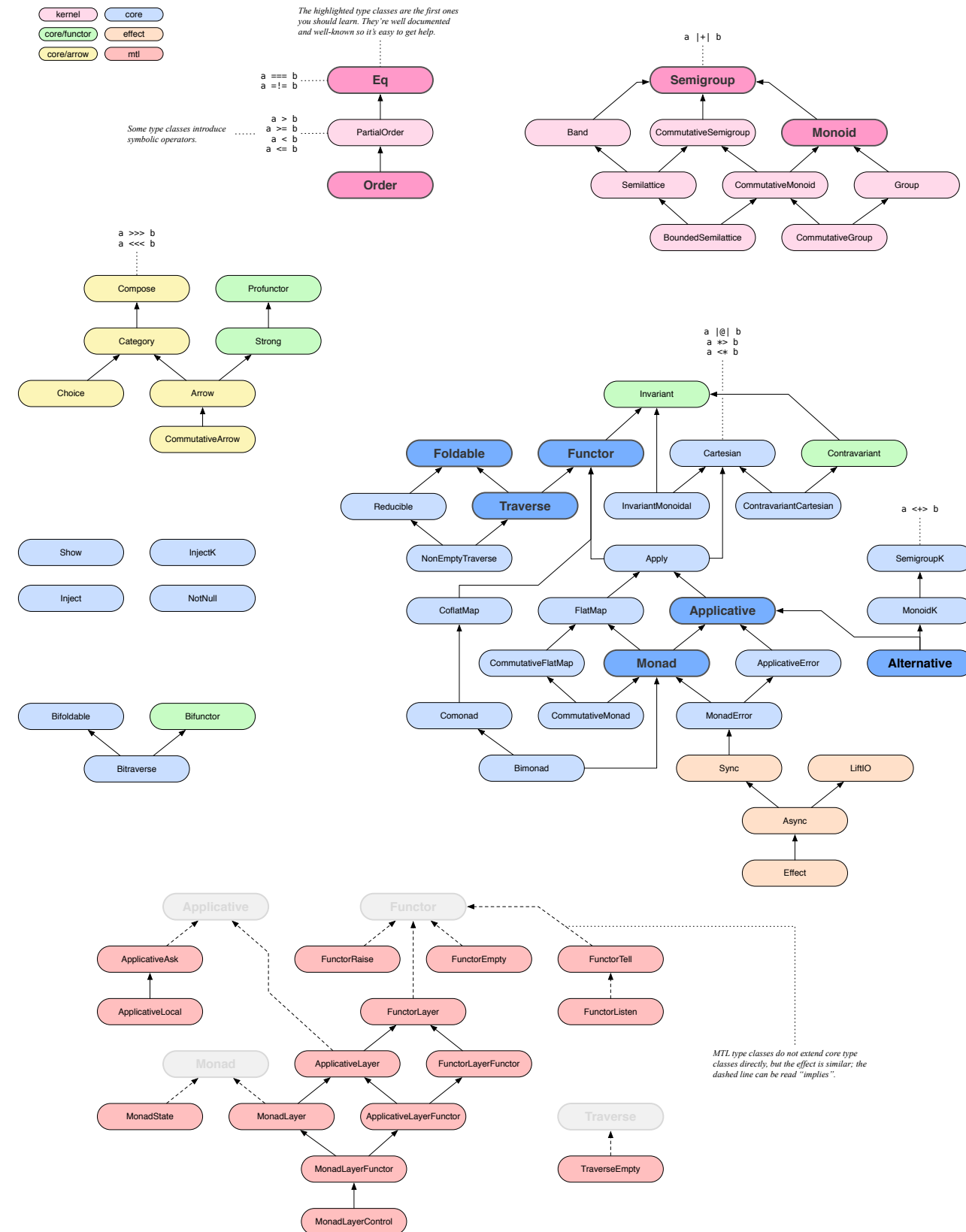
MONOID >> combining 2 values into 1

FUNCTOR >> values lifted to a context

APPLICATIVE >> independent values applied
to a function in a context

MONAD >> ops in sequence in a context

kernel	core
core/functor	effect
core/arrow	mtl



FORGET ABOUT THE DETAILS
FOCUS ON
HOW THINGS COMPOSE

WANNA KNOW MORE?

- > CATEGORY THEORY FOR THE WH BY @PHILIPWADLER
- > CATEGORY THEORY BY @BARTOSZMILEWSKI
- > CATS-INFGRAPHICS BY ROB NORRIS - @TPOLECAT
- > CATS DOCUMENTATION – TYPE CLASSES

THANK YOU!

- > **TWITTER:** @DANIELASFREGOLA
- > **BLOG:** DANIELASFREGOLA.COM

github.com/DanielaSfregola/tutorial-cat